Part 1: Docker Basics

1. What is Docker?

Docker is a tool that packages an application and everything it needs to run into a container.

Why do we use Docker?

Without Docker: Developer Machine -> "It works on my machine" -> Test Environment -> Dependency/Configuration issues -> Production Failure.

Docker packages Application + Dependencies + Libraries + Runtime + Configuration into a Docker Image, which runs as a Container. The same image can be deployed across environments.

2. Docker Architecture

Docker follows a client-server architecture.

```
Docker Client
    | Docker API
    v
Docker Daemon (dockerd) --> Images / Containers / Networks / Volumes
    |
    v
containerd -> runc -> Linux Kernel -> Container
```

3. Docker Client

The Docker Client is what we interact with, mostly the Docker CLI: `docker ps`, `docker images`, `docker build`, `docker run`, `docker pull`, `docker push`, `docker stop`.

The CLI sends the request to the Docker daemon through the Docker API.

> **Interview answer:** "Docker Client is the command-line interface through which users interact with Docker. It sends Docker commands to the Docker daemon using the Docker API."

4. Docker Daemon (dockerd)

The Docker daemon is the main background service. It listens for Docker API requests and manages Containers, Images, Networks and Volumes.

Flow for `docker run nginx`: CLI -> Daemon -> Check image -> Create container -> Start container.

> **Interview answer:** "Docker daemon, or dockerd, is a background service responsible for managing Docker objects such as images, containers, networks, and volumes."

5. Docker Images

A Docker image is a read-only template used to create containers. Examples: `nginx:latest`, `postgres:16`, `ubuntu:24.04`.

An image contains the application, libraries, dependencies, configuration, filesystem and runtime requirements.

`docker pull nginx` downloads the Nginx image from a registry.

6. Docker Container

A container is a running instance of an image.

- Image = Blueprint
- Container = Running instance of that blueprint

```
docker run -d --name web nginx
  nginx  -> image
  web    -> container name
  -d     -> detached mode
```

One Nginx image can create many containers (Container 1, 2, 3...).

7. Docker Registry

A Docker Registry stores Docker images. Examples: Docker Hub, Google Artifact Registry, Amazon ECR, Azure Container Registry, private registries.

Project example (Google Artifact Registry):

```
us-central1-docker.pkg.dev/project-7e1069a7-5b78-49e8-9b8/quickstart-docker-repo/employment-management:3
    8
```

Flow: Developer -> docker build -> Docker Image -> docker push -> Google Artifact Registry -> Kubernetes -> Container.

8. Docker Runtime

Docker CLI -> dockerd -> containerd -> runc -> Linux Kernel.

- containerd: manages container lifecycle (create, start, stop, manage images).
- runc: low-level runtime that actually creates and starts containers per the OCI runtime specification.

9. Complete Architecture Flow

```
Developer -> Docker CLI -> Docker API -> dockerd (Docker Daemon)
dockerd -> Images / Containers / Networks
dockerd -> containerd -> runc -> Linux Kernel -> Container
```

10. Docker Build Process

For a Java Spring Boot app, create a Dockerfile:

```
FROM eclipse-temurin:17-jre
COPY target/employment-management-1.0.0.jar app.jar
EXPOSE 8180
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
docker build -t employment-management:1.0 .
docker run -p 8180:8180 employment-management:1.0
```

Flow: Dockerfile -> docker build -> Image -> docker run -> Container -> Application -> Port 8180.

11. Docker vs Virtual Machine

- Virtual Machine: Physical Server -> Hypervisor -> VM (each with its own complete Guest OS + Application).
- Docker: Physical Server -> Host OS -> Docker Engine -> Containers (App1, App2, App3).

Containers share the host kernel, so they are generally: lightweight, faster to start, more efficient in resources, easy to package and deploy.

12. Real-Time DevOps Example

> **Interview answer:** "In my project, I containerized a Spring Boot application using Docker. First, we package the application as a JAR using Maven. Then we use a Dockerfile to create a Docker image. The image is tagged with the application version and pushed to Google Artifact Registry. Kubernetes then pulls that image from the registry and creates containers from it. This allows us to use the same application image across different environments."

CI/CD flow: Developer -> GitHub -> Jenkins -> Maven Build -> Unit/Integration Tests -> Docker Build -> Docker Image -> Google Artifact Registry -> Kubernetes / GKE -> Deployment -> Pod -> Container -> Spring Boot Application.

Basic Docker Commands

Command	Simple meaning
docker version	Shows Docker client and server/version information
docker info	Shows detailed information about Docker
docker pull	Downloads an image from a Docker registry
docker create	Creates a container but does not start it
docker rename	Changes the name of a container
docker start	Starts an existing stopped container
docker stop	Stops a running container
docker rm	Removes/deletes a container
docker rmi	Removes/deletes a Docker image
docker run	Creates and starts a new container

Part 2: Dockerfile

What is a Dockerfile?

A Dockerfile is a text file with instructions for Docker to build a Docker image. Think of it as a recipe: Dockerfile -> docker build -> Image -> docker run -> Container.

```
FROM ubuntu:24.04
WORKDIR /app
COPY app.jar /app/
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

Dockerfile Instructions

1. FROM

Specifies the base image. Almost every Dockerfile starts with FROM. Example: `FROM eclipse-temurin:17-jre`.

> **Interview answer:** "FROM defines the base image for the Docker image."

2. MAINTAINER (deprecated)

Traditionally specified the image maintainer. Use `LABEL maintainer="Manjunath"` instead.

> **Interview answer:** "MAINTAINER was used to specify the image maintainer, but it is deprecated and LABEL is preferred."

3. WORKDIR

Defines the working directory inside the container. `WORKDIR /app` then `COPY app.jar .` puts the file at /app/app.jar.

> **Interview answer:** "WORKDIR sets the working directory inside the container."

4. RUN

Executes a command while building the image (`RUN apt-get update`, `RUN apt-get install -y curl`). RUN -> image build time.

> **Interview answer:** "RUN executes commands during the Docker image build process."

5. ENV

Defines environment variables inside the image/container. `ENV APP_PORT=8080`; `echo $APP_PORT` prints 8080.

> **Interview answer:** "ENV is used to set environment variables in the Docker image and container."

6. ARG

Defines a build-time variable: `ARG VERSION=17`, then `docker build --build-arg VERSION=21 -t myapp .`

ARG -> build time; ENV -> available in image/container environment.

7. LABEL

Adds metadata (maintainer, version, description) to an image. It doesn't normally affect how the app runs.

> **Interview answer:** "LABEL is used to add metadata such as version, maintainer, or description to a Docker image."

8. COPY

Copies files from the build context into the image: `COPY app.jar /app/app.jar`.

> **Interview answer:** "COPY is used to copy files and directories from the build context into the Docker image."

9. ADD

Similar to COPY but with extras, such as automatically extracting local tar archives (`ADD app.tar /app/`). For normal copying prefer COPY.

> **Interview answer:** "ADD copies files into the image and also supports additional features such as extracting local tar archives. For simple copying, COPY is generally preferred."

10. CMD

Specifies the default command when a container starts: `CMD ["java", "-jar", "app.jar"]`. RUN -> during image build; CMD -> when container starts.

> **Interview answer:** "CMD specifies the default command that runs when a container starts."

11. ENTRYPOINT

Defines the main executable for the container. CMD can easily be overridden by a command given to `docker run`; ENTRYPOINT defines the main executable. They can be combined:

```
ENTRYPOINT ["java", "-jar"]
CMD ["app.jar"]
Result: java -jar app.jar
```

> **Interview answer:** "ENTRYPOINT defines the main executable of the container, while CMD provides default arguments or a default command that can be overridden."

12. USER

Specifies the user for subsequent commands and/or the container process (`USER appuser`). Important for security: avoid running as root unnecessarily.

> **Interview answer:** "USER specifies the user under which commands and the container's main process run."

13. EXPOSE

Documents the port the app listens on (`EXPOSE 8080`). It does not publish the port; use `docker run -p 8080:8080 myapp`.

> **Interview answer:** "EXPOSE documents the port on which the containerized application listens. It does not publish the port; port publishing is done using the -p option of docker run."

14. ONBUILD

Defines an instruction that executes when another Dockerfile uses this image as its base (`ONBUILD COPY . /app`). Mainly for reusable base images.

15. SHELL

Specifies the shell used by shell-form commands such as RUN: `SHELL ["/bin/bash", "-c"]`.

> **Interview answer:** "SHELL specifies the default shell used to execute shell-form commands in the Dockerfile."

16. STOPSIGNAL

Specifies which system signal Docker sends to stop the container (`STOPSIGNAL SIGTERM`). SIGTERM is the default.

17. HEALTHCHECK

Tells Docker how to check whether the app is healthy: `HEALTHCHECK CMD curl --fail http://localhost:8080/health || exit 1`. Docker shows: healthy / unhealthy / starting (check with `docker ps`).

> **Interview answer:** "HEALTHCHECK is used to determine whether the application inside a container is healthy and responding correctly."

18. VOLUME

Defines a location for persistent data: `VOLUME /var/lib/postgresql/data`. Data survives container replacement.

Easy Way to Remember (focus for interviews)

```
FROM        -> Base image
WORKDIR     -> Working directory
RUN         -> Build-time command
COPY        -> Copy files
ADD         -> Copy + additional features
ENV         -> Environment variable
ARG         -> Build-time variable
EXPOSE      -> Document container port
CMD         -> Default command
ENTRYPOINT  -> Main executable
USER        -> Run as specific user
VOLUME      -> Persistent data
HEALTHCHECK -> Check application health
```

Docker Tag vs Docker Digest (SHA256)

Tag	Digest (SHA256)
Human-friendly name/version, e.g. myapp:1.0	SHA256 hash, e.g. myapp@sha256:abc123...
Easy to remember	Long and difficult to remember
Can be changed/moved	Identifies the exact image content
Does not guarantee the exact image	Guarantees the exact image content
Good for development and CI/CD versioning	Good for production, security and reproducible deployments

Part 3: Multi-Stage Builds

19. What is a Docker multi-stage build?

A multi-stage build uses multiple FROM instructions in a single Dockerfile. Each stage does a different task (compile, test, prepare the final image). Only the required artifacts are copied into the final image, leaving behind build tools and intermediate files.

- Stage 1 Build: source code + Maven + JDK, compile and package, copy the JAR.
- Stage 2 Runtime: JRE + application JAR, run the application.

Why use multi-stage builds?

- Reduce the final image size.
- Avoid including compilers, Maven and other build tools in production.
- Reduce unnecessary packages and attack surface.
- Keep Dockerfiles organized around build and runtime tasks.

Single-stage vs multi-stage

Single-stage	Multi-stage
One build stage	Two or more stages
Build tools may remain in the final image	Build tools can be excluded
Can produce larger images	Can produce smaller images
Build and runtime concerns combined	Build and runtime concerns separated

Note: a well-optimized single-stage build can also be small; multi-stage makes the separation easier.

Multi-stage build with a real-time Java application

```
# Stage 1: Build the application
FROM maven:3.9-eclipse-temurin-21 AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Run the application
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- FROM ... AS builder: creates and names the build stage.
- COPY --from=builder: copies the JAR from the builder stage into the runtime stage.
- ENTRYPOINT: starts the Spring Boot application.

Real-time scenario: image is 1.2 GB, how to reduce it?

Sample answer: "I would inspect the current Dockerfile and identify unnecessary build tools, caches, and files. I would use a multi-stage build to compile the Java application in a Maven/JDK image, then copy only the executable JAR into a suitable JRE runtime image. I would also review the base image, Docker build context, and dependency layers, then compare the resulting image sizes and run tests to confirm that the application works correctly."

Quick revision

Instruction	Purpose
FROM	Selects a base image and begins a stage

Instruction	Purpose
AS builder	Names a build stage
RUN	Executes a command during the image build
COPY --from	Copies artifacts from another stage
--target	Selects a stage to build
ENTRYPOINT	Defines the main container executable

Part 4: STOPSIGNAL

What is STOPSIGNAL in Docker?

STOPSIGNAL specifies which OS signal Docker sends to a container's main process when it is stopped. Default: usually SIGTERM.

`docker stop mycontainer` sends SIGTERM; if the process doesn't stop within the timeout, Docker sends SIGKILL.

Signal	Meaning
SIGTERM	Requests graceful shutdown
SIGINT	Interrupt signal
SIGQUIT	Requests termination; behavior depends on the application
SIGKILL	Forces immediate termination; cannot be caught or ignored

STOPSIGNAL doesn't create a multi-stage build; it controls shutdown behavior and works in single or multi-stage Dockerfiles.

STOPSIGNAL in a multi-stage Dockerfile

Configure it in the final runtime stage, because the final stage's image configuration determines the container's stop signal.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
STOPSIGNAL SIGTERM
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

STOPSIGNAL Scenario Questions

Q1. The app takes time to save data before shutting down. How do you handle it?

Use `STOPSIGNAL SIGTERM`, make sure the app handles SIGTERM gracefully, and configure a stop timeout: `docker stop --timeout=30 myapp`. The signal requests graceful shutdown; the application must implement the shutdown behavior.

Q2. You set STOPSIGNAL SIGTERM but the container still gets killed. Why?

- The app doesn't handle SIGTERM correctly.
- The app takes longer than the stop timeout.
- The main process is not forwarding signals to the application.
- The app is stuck or cannot finish its cleanup.

Solution: check logs (`docker logs myapp`), verify the main process, handle signals properly, increase the timeout if needed.

Q3. STOPSIGNAL set in Stage 1: does it apply to the final image?

No, not automatically. The final stage's own configuration determines the stop signal.

Q4. Nginx: how to customize the shutdown signal?

`FROM nginx:alpine` + `STOPSIGNAL SIGQUIT`. For Nginx, SIGQUIT requests graceful shutdown while SIGTERM requests termination. Test with your own image and configuration.

Q5. Docker sends SIGTERM but the container does not stop. What next?

When the stop timeout expires, Docker sends SIGKILL. Inspect with:

```
docker inspect --format='{{.Config.StopSignal}}' myapp
docker inspect --format='{{.State.ExitCode}}' myapp
```

The exit code alone does not prove why the process exited.

Q6. Multi-stage image works, but shutdown is unreliable in Kubernetes. What do you investigate?

- Final image's configured stop signal.
- Whether the app receives termination signals.
- App is the main process (prefer exec-form ENTRYPOINT).
- App's graceful-shutdown configuration.
- Kubernetes termination grace-period settings.
- Logs for unfinished requests or pending DB operations.

Q7. STOPSIGNAL vs CMD vs ENTRYPOINT vs EXPOSE

Instruction	Purpose
STOPSIGNAL	Defines the signal used to request container shutdown
CMD	Defines the default command or arguments
ENTRYPOINT	Defines the executable to run
EXPOSE	Documents the port the application listens on

Q8. Small production image + graceful shutdown: how to design it?

Multi-stage build, keep build dependencies out of the runtime image, use an appropriate runtime base, configure the shutdown signal, and make sure the app handles shutdown.

```
docker build -t demo-java-app .
docker run -d --name demo-app -p 8080:8080 demo-java-app
docker inspect --format='{{.Config.StopSignal}}' demo-app
docker stop --timeout=30 demo-app
docker logs demo-app
```

Part 5: HEALTHCHECK

1. What is a Docker health check?

The HEALTHCHECK instruction tells Docker how to test whether the app inside a container is working. A container can be running but unhealthy.

- Starting: the check has not yet succeeded during startup.
- Healthy: the check is passing.
- Unhealthy: repeated checks have failed.

2. Why do we need a health check?

It detects apps that are running but not functioning, monitors readiness, and gives monitoring systems a status to act on.

Interview point: a health check does not automatically restart an unhealthy container. Restart policies normally respond to the process exiting, not to health becoming unhealthy.

3. Health check in a multi-stage build

Configure it in the final runtime stage, because that is the image used to run the app.

4. Practical example: Spring Boot + multi-stage

Assumes port 8080 and the Actuator endpoint `http://localhost:8080/actuator/health`, which returns `{"status": "UP"}`.

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Runtime
FROM eclipse-temurin:21-jre
WORKDIR /app
# Install curl so the health check can make HTTP requests
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s \
  --start-period=60s --retries=3 \
  CMD curl --fail --silent http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Health check options

Option	Meaning
--interval=30s	Run the check every 30 seconds
--timeout=5s	A check taking longer than 5 seconds counts as failed
--start-period=60s	Allow time for initial application startup
--retries=3	Unhealthy after three consecutive failures
CMD	The command Docker executes; --fail makes HTTP errors like 500 fail

To verify database health too, configure Spring Boot's health endpoint to include it.

Run and inspect

```
docker build -t student-app:1.0 .
docker run -d --name student-app -p 8080:8080 student-app:1.0
docker ps
docker inspect --format='{{json .State.Health}}' student-app
docker logs student-app
docker rm -f student-app
```

5. Health check scenario questions

Q1. Container is running but the app is not responding. How to troubleshoot?

Check `docker ps`, `docker logs student-app`, and `docker inspect --format='{{.State.Health.Status}}' student-app`. If unhealthy, inspect recent health output and verify the endpoint from inside the container. Takeaway: running does not mean healthy.

Q2. Container becomes unhealthy during startup. How to fix it?

The app needs 45 s to start but checks fail after 10 s. Configure a suitable `--start-period` and interval (e.g. `--start-period=60s --interval=30s --timeout=5s --retries=3`). Don't make the grace period too short.

Q3. Container becomes unhealthy because the database is down?

Check DB connection and logs, configure a health endpoint that reflects DB connectivity, verify hostname/port/credentials/network, and decide whether DB unavailability should make the app unhealthy. A simple TCP check on 8080 may pass even if the DB is down.

Q4. Image builds fine but health is always unhealthy. What to check?

- Is the app listening on the expected port?
- Is the endpoint path correct?
- Does the final image contain curl?
- Does the endpoint return a successful HTTP response?
- Is the app still starting?
- Is the health command syntax correct?

`docker exec student-app curl -v http://localhost:8080/actuator/health` (needs curl and a running container).

Q5. Explain the complete health-check lifecycle

1) Docker starts the container. 2) App initializes. 3) Docker runs the check at intervals. 4) Success -> healthy; consecutive failures -> unhealthy. 5) Status is visible via `docker ps` / `docker inspect`. 6) An operator, monitor or orchestrator responds. The health check monitors; it does not by itself guarantee recovery.

Part 6: Dangling Images

1. Dangling Docker images

Q1. What are dangling images?

Untagged images not referenced by a tag, shown as `<none>`. They commonly appear when rebuilding an image and the older one loses its tag.

Q2. Why remove them?

To reclaim disk space and keep the local cache tidy. Dangling does not always mean useless (build cache, recovery).

Q3. How do they work / commands

```
docker images
docker images -f dangling=true
docker image prune        # remove dangling images
docker image prune -a     # remove ALL unused images (be careful)
```

Q4. Linux VM running out of disk because of Docker?

`docker system df`, `docker images`, `docker image prune`, `docker container prune`. Inspect first; don't blindly run destructive cleanup on production.

Q5. Does deleting a dangling image delete a running container?

Normally no. Docker protects images still in use.

Part 7: Multi-Stage Builds with Python

2. Multi-stage builds with Python

```
FROM python:3.12-slim AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=build /install /usr/local
COPY app.py .
CMD ["python", "app.py"]

docker build -t python-app .
docker run --rm python-app
```

Q4. Python image is 1 GB. How to optimize?

Inspect layers, use a slim base, avoid unnecessary packages and caches, use multi-stage where useful, then test.

Q5. What does COPY --from=build do?

Copies selected files from the build stage into the runtime stage, not the entire stage.

Q6. Is multi-stage always smaller?

No. Measure with `docker images` and inspect layers with `docker history`.

Part 8: Docker Buildx and BuildKit

3. Docker Buildx

Buildx is a Docker CLI plugin that provides advanced build features using BuildKit: multi-architecture builds, advanced caching, build secrets, multi-platform publishing.

```
docker buildx version
docker buildx ls
docker buildx inspect
docker buildx build -t myapp:1.0 .
docker buildx build --platform linux/amd64,linux/arm64 -t username/myapp:1.0 --push .
```

Q4. Intel and ARM servers: compatible image?

Use Buildx with `--platform linux/amd64,linux/arm64`, confirm dependencies support both, publish to a registry.

Q5. Buildx succeeds but docker images doesn't show the image?

It may be in the BuildKit cache or pushed to a registry. For a local single-platform build use `docker buildx build --load -t myapp:1.0 .`

4. Docker build, new ways

Q1. What is BuildKit?

Docker's modern build engine: efficient caching, parallel execution, secret mounts. BuildKit is the engine; Buildx is the CLI interface.

Q3. Build commands

```
docker build -t myapp:1.0 .
docker build --no-cache -t myapp:1.0 .    # ignore cache
docker build --pull -t myapp:1.0 .        # check for newer base image
```

Q4. What is the build context?

The set of files made available to the builder (in `docker build -t myapp .` the current directory). Use `.dockerignore`: `.git`, `node_modules`, `.venv`, `__pycache__`, `*.log`, `.env`. Excluding .env is not a replacement for secrets management.

Q5. Every small code change causes a 10-minute build?

Order Dockerfile instructions for cache reuse: copy dependency manifests first, install dependencies, then copy frequently changing source.

Q6. Pass a build secret without embedding it in the image?

Use a BuildKit secret mount:

```
# syntax=docker/dockerfile:1
FROM alpine
RUN --mount=type=secret,id=token \
    test -s /run/secrets/token
```

`docker build --secret id=token,src=token.txt -t secret-demo .`

Part 9: Distroless and Scratch

5. Distroless images

A Distroless image contains the app runtime and required dependencies, without a typical shell, package manager or general-purpose OS utilities. Benefits: less software in production, smaller attack surface, no build or troubleshooting tools.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM gcr.io/distroless/java21-debian13:nonroot
WORKDIR /app
COPY --from=build /app/target/*.jar /app/app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Verify the image tag and Java compatibility before using it.

Q4. docker exec -it myapp sh fails. Why?

Distroless generally has no shell. Troubleshoot via `docker logs`, `docker inspect`, `docker stats`, or a compatible debug image.

Q5. Does Distroless eliminate all vulnerabilities?

No. Code, runtime and dependencies can still be vulnerable; scanning and patching remain necessary.

6. Scratch vs Distroless

scratch is an empty starting point with no shell, libraries or utilities.

Feature	Scratch	Distroless
Initial contents	Empty	Runtime and required components
Shell	No	Generally no
Package manager	No	No general-purpose one
Common use	Self-contained binaries	Apps needing supported runtimes
Setup effort	Often higher	Often easier for supported languages

- Java Spring Boot: normally Distroless Java (needs a Java runtime).
- Statically linked Go binary: Scratch can be suitable (include certificates if needed).
- Troubleshooting Scratch with a shell: usually not possible; use logs, inspection, external monitoring.

Part 10: Image Scanning

7. Image scanning tools

Image scanning checks images for known vulnerabilities in OS packages and app dependencies, and sometimes secrets or misconfigurations. Tools: Trivy, Docker Scout, Grype. Example: `trivy image nginx:latest`.

Q4. Jenkins pipeline detects a critical vulnerability?

Identify the package and CVE, check for a patched version, update base image/dependency, rebuild, scan again, follow the release policy.

Q5. Scan reports nothing. Is the image completely secure?

No. Scanners can't detect everything; follow secure configuration, manage secrets, update dependencies, monitor the running app.

Q6. Include scanning in CI/CD?

Add a scanning stage after build and before publish/deploy; fail the pipeline on findings that violate the severity policy.

Part 11: Storage and Networking

8. Docker container storage

Docker uses image layers and container writable layers, plus volumes and bind mounts for persistent data and host file sharing.

Docker volume	Bind mount
Managed by Docker	Uses a specific host path
Good for persistent application data	Good for source code and host files
More portable between host layouts	Depends on the host path

```
docker volume create postgres-data
docker volume ls
docker volume inspect postgres-data
docker run -d --name postgres-db -e POSTGRES_PASSWORD='ChangeMe123!' \
  -v postgres-data:/var/lib/postgresql/data postgres:16
```

This is a lab example, not a production secret configuration.

- Deleted a DB container: data in an intact named volume normally survives; data only in the writable layer may be lost.
- Disk usage: `docker system df`, `docker system df -v`, `docker images`, `docker volume ls`. Verify backups before cleaning persistent data.
- `docker compose down -v` removes containers, networks and can remove named volumes. Avoid -v when DB data must be retained.

9. Docker networking

Docker networking enables communication between containers, the host and external systems.

Driver	Purpose
bridge	Connects containers on one Docker host
host	Shares the host network namespace on supported Linux systems
none	Disables normal container network connectivity
overlay	Supports networking across Swarm nodes

```
docker network create app-network
docker run -d --name web --network app-network nginx
docker run --rm --network app-network alpine wget -qO- http://web
```

On a user-defined bridge network, Docker provides name resolution between containers.

- Backend can't reach PostgreSQL: `docker ps`, `docker network ls`, `docker network inspect app-network`, `docker logs backend`. Check shared network, hostname, port, readiness, credentials.
- Why not use localhost to reach another container? localhost inside the backend refers to the backend container itself. Use the service name.
- EXPOSE 8080 documents the container port; `-p 8080:8080` publishes a host port to the container port.

Part 12: Docker Compose

10. Docker Compose

Docker Compose defines and runs multi-container applications using a YAML file, usually `compose.yaml`. Define images, ports, networks, environment variables, volumes and dependencies in one place (frontend + backend + PostgreSQL).

Useful commands

```
docker compose config      # validate and render config
docker compose build
docker compose up -d       # start in background
docker compose ps
docker compose logs -f     # follow logs
docker compose restart
docker compose down        # remove containers and networks
```

- Backend starts before PostgreSQL is ready: add a PostgreSQL health check, use `depends_on` with `condition: service_healthy`, and implement connection retries.
- `up` runs in the foreground and shows logs; `up -d` runs in the background.
- Services on the same Compose network communicate by service name.

Part 13: Docker Swarm

11. Docker Swarm

Docker's built-in orchestration system for managing services across multiple Docker hosts.

- Run multiple replicas, schedule containers across nodes.
- Service discovery and load balancing, rolling updates.
- Reconciles tasks toward the desired replica count.

Manager nodes maintain cluster state and schedule tasks; worker nodes run tasks.

```
docker swarm init
docker node ls
docker service create --name web --replicas 3 --publish 8080:80 nginx
docker service ls
docker service ps web
docker service logs web
docker service scale web=5
```

- One of three replicas fails: Swarm attempts to restore the desired count; check task status and logs.
- Worker goes down: manager may schedule replacement tasks on other eligible nodes, depending on capacity and constraints.

Compose vs Swarm

Docker Compose	Docker Swarm
Defines multi-container applications	Orchestrates services across a cluster
Commonly for local development	Cluster-level deployment
docker compose up	docker service commands
No multi-node orchestration itself	Schedules tasks across Swarm nodes

Part 14: Compose Examples

12. Compose example 1: Python app with PostgreSQL

Scenario: start a Python app and PostgreSQL with one command and preserve DB data.

```
python-project/
  app.py
  requirements.txt
  compose.yaml
```

requirements.txt: `flask`, `psycopg2-binary`

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Python application is running!"

app.run(host="0.0.0.0", port=5000)
```

compose.yaml:

```
services:
  app:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 10s
      timeout: 5s
      retries: 5
volumes:
  postgres-data:
```

Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

Create a `.env` file with `DB_PASSWORD=ReplaceWithALabPassword`, then:

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs -f
```

Open http://localhost:5000. This example does not yet connect Flask to PostgreSQL. Keep .env out of Git; use proper secrets management in production.

- Why a named volume? To preserve DB data when the container is recreated.
- Why listen on 0.0.0.0? So the server accepts connections via the container's network interfaces; 127.0.0.1 would block access through the published port.

- Why condition: service_healthy? It delays the dependent service until the DB health check succeeds; the app should still handle DB failures later.

13. Compose example 2: Java Spring Boot with PostgreSQL

```
spring-project/
  Dockerfile
  compose.yaml
  pom.xml
  src/
```

Dockerfile (multi-stage):

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

compose.yaml:

```
services:
  backend:
    build: .
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/studentdb
      SPRING_DATASOURCE_USERNAME: student
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: studentdb
      POSTGRES_USER: student
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - student-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U student -d studentdb"]
      interval: 10s
      timeout: 5s
      retries: 5
volumes:
  student-data:
```

Create a `.env` with DB_PASSWORD. Make sure the project includes the PostgreSQL JDBC driver.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs -f backend
docker compose logs -f db
```

- Spring Boot fails with a DB error: check DB logs and health, hostname is `db` not localhost, port, credentials, JDBC driver, datasource config, retries.
- Updated Java code: `docker compose up -d --build backend`.
- Confirm it works: check status, logs, test an endpoint, Actuator health.
- Stop without removing the DB volume: `docker compose down`.

Part 15: Interview Scenarios (Compose)

Scenario 1: Backend starts before the database

Answer: configure a PostgreSQL health check, use `depends_on` with `condition: service_healthy`, and implement DB connection retries. depends_on alone does not mean the DB is ready for connections.

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: example123
      POSTGRES_DB: appdb
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 10s
      timeout: 5s
      retries: 5
  backend:
    image: my-backend:1.0
    depends_on:
      db:
        condition: service_healthy
```

Scenario 2: Backend cannot connect to PostgreSQL

Check DB container status, logs, network, hostname, port, credentials: `docker compose ps`, `docker compose logs db`, `docker compose logs backend`, `docker network ls`. Use the service name (`jdbc:postgresql://db:5432/appdb`), never localhost.

Scenario 3: Database data disappears after restart

Data may not be in a persistent volume, or the volume was removed or mounted incorrectly. Check `docker volume ls` and `docker volume inspect postgres-data`. `docker compose down` normally retains named volumes; `down -v` can remove them.

Scenario 4: Developer changes the application code

Rebuild and recreate: `docker compose up -d --build backend`, then `docker compose ps` and `docker compose logs -f backend`. Host source changes don't update an already-built image unless a bind mount or hot-reload is configured.

Scenario 5: A container exits immediately

Check `docker compose ps -a`, `docker compose logs backend`, `docker inspect <container_name>`. Causes: app startup failure, wrong ENTRYPOINT/CMD, missing environment variables, failed DB connection, main process exits normally. A container stays running only while its main process runs.

Scenario 6: One service consumes too many resources

Use `docker stats` and `docker compose ps`, review app memory settings, configure container resource limits where supported, and verify they are enforced by the runtime.

Scenario 7: Troubleshooting a Compose app quickly

`docker compose config`, `docker compose ps`, `docker compose logs --tail=100`, `docker network inspect <network_name>`. Then verify the frontend's backend URL, port mappings, DNS resolution and app config.

Scenario 8: Same app on different developers' machines

Compose gives a repeatable configuration: `docker compose up -d --build`. Keep environment-specific settings outside the shared config and avoid committing credentials to Git.

Part 16: Interview Scenarios (Swarm)

Scenario 9: One replica crashes in production

Swarm tries to reconcile to the desired state and replace failed tasks. Use `docker service ls`, `docker service ps myapp`, `docker service logs myapp`. An unhealthy-but-running container is not necessarily restarted just because of health status.

Scenario 10: Scale from 3 replicas to 10

`docker service scale myapp=10`, `docker service ls`, `docker service ps myapp`. Follow-up: scaling doesn't always make the app faster; the bottleneck may be DB, CPU, memory, network or an external dependency.

Scenario 11: A worker node goes down

The manager detects it and may schedule replacement tasks on other eligible nodes if resources allow. Check `docker node ls` and `docker service ps myapp`. Recovery is not guaranteed without capacity or if constraints exclude nodes.

Scenario 12: Deploy across three servers (Compose or Swarm?)

Use Swarm for native multi-node orchestration; Compose alone does not schedule across nodes.

```
docker swarm init
docker swarm join --token <worker-token> <manager-ip>:2377
docker node ls
```

Treat join tokens as credentials.

Scenario 13: Service cannot start all replicas (10 requested, 6 running)

Check `docker service ps myapp`, `docker node ls`, `docker service inspect myapp`. Causes: not enough CPU/memory, unavailable workers, registry auth problems, placement constraints, app startup failures.

Scenario 14: Update with minimal downtime

Use a controlled rolling update:

```
docker service update --image nginx:1.28 \
  --update-parallelism 1 --update-delay 10s myapp
```

If the new version fails, inspect tasks and logs, then `docker service rollback myapp`.

Scenario 15: Restrict where a service runs

```
docker node update --label-add role=database worker1
docker service create --name db \
  --constraint 'node.labels.role==database' postgres:16
```

A placement constraint is not a complete security boundary; also use network isolation, access controls and secret management.

Scenario 16: Published Swarm service cannot be accessed

Verify service status, port publishing, firewall rules, listening ports and network config (`docker service ls`, `docker service ps myapp`, `docker service inspect myapp`). Check required Swarm ports and host/cloud firewalls. A running task doesn't guarantee correct external access.

Part 17: Advanced Compose vs Swarm

Scenario 17: Can Compose restart a failed container?

Compose can set restart policies such as `restart: unless-stopped`, but they respond to container termination, not to an unhealthy status alone. In Swarm, the orchestrator maintains the desired replica count.

Scenario 18: Can Compose provide high availability across hosts?

No. Standard `docker compose up` is not a multi-node orchestrator. Consider Swarm or another orchestration platform.

Scenario 19: What if the Swarm manager fails?

With other managers and a majority quorum, the cluster continues. A single-manager cluster that loses its manager keeps existing tasks running, but management and scheduling are impaired. For production use an odd number of managers (3 or 5).

Scenario 20: Securely managing passwords in Compose and Swarm

Avoid hardcoding passwords in Dockerfiles, source control or committed Compose files. Use an approved secrets manager or supported mechanisms, limit access, rotate credentials, avoid logging secrets. In Swarm, use Docker secrets mounted as files.

Part 18: Quick Interview Challenge

Try answering these first

- Compose starts the database container, but the backend fails immediately.
- A Swarm service needs 8 replicas, but only 5 are running.
- You want database data to survive container recreation.
- Your service update introduces a serious error.
- A backend uses localhost to connect to PostgreSQL in another container.
- A container is unhealthy but has not restarted.
- A Swarm worker goes offline.
- You need one command to start a local frontend, backend and database.

What you should remember for the interview

- Compose: defines and runs a multi-container application.
- Swarm: orchestrates services across multiple Docker nodes.
- Compose health checks: help determine whether a service is ready.
- Swarm replicas: the desired number of service tasks.
- Persistent volumes: protect data from ordinary container recreation.
- Troubleshooting: start with status, logs, inspection, networking and resource usage.